

B1-2 시스템 장애 분석 리포트

OOM Crash, CPU Latency, Deadlock, Scheduling Analysis

Evidence root: B1-2/evidence/

Generated: 2026-06-04 15:37:05

Linux Docker 환경에서 제공 바이너리를 실행하고, 로그와 표준 프로세스 도구 출력만 사용했습니다.

[Bug] OOM Crash - MemoryGuard가 메모리 누수성 증가를 감지해 프로세스를 강제 종료

1. Description (현상 설명)

agent-leak-app을 MEMORY_LIMIT=50으로 실행하면 부트 시퀀스는 모두 통과하지만, 워크로드 시작 후 약 5초 안에 프로세스가 종료된다. 실행 로그에는 MemoryWorker의 Heap 값이 25MB에서 50MB로 증가한 직후 MemoryGuard가 임계치 초과를 감지하고 self-termination을 수행한 기록이 남았다.

비교 실행으로 MEMORY_LIMIT=128을 적용하면 동일한 메모리 증가 패턴이 더 오래 지속되며, 150MB 시점에서 128MB 제한을 초과해 종료된다. 따라서 종료 원인은 OS의 불특정 OOM Killer가 아니라 애플리케이션 내부의 MemoryGuard 정책으로 판단된다.

2. Evidence & Logs (증거 자료)

Evidence:

- Before: B1-2/evidence/oom-before/20260604-062049/
- After: B1-2/evidence/oom-after/20260604-061950/

Before 실행 조건:

```
MEMORY_LIMIT=50
CPU_MAX_OCCUPY=100
MULTI_THREAD_ENABLE=false
```

Before 실행 로그 핵심 구간:

```
2026-06-04 06:20:51,748 [INFO] [MemoryWorker] Current Heap: 25MB
2026-06-04 06:20:54,775 [INFO] [MemoryWorker] Current Heap: 50MB
2026-06-04 06:20:54,776 [CRITICAL] [MemoryGuard] Memory limit exceeded (50MB >= 50MB) /
(Recommend Over 256MB)
2026-06-04 06:20:54,776 [CRITICAL] [MemoryGuard] Self-terminating process 39 to prevent
system instability.
Killed
```

Before monitor.csv 요약:

```
first=17104KB at 2026-06-04 06:20:50
last=42708KB at 2026-06-04 06:20:53
max=42708KB at 2026-06-04 06:20:52
```

After 실행 조건:

```
MEMORY_LIMIT=128
CPU_MAX_OCCUPY=100
```

```
MULTI_THREAD_ENABLE=false
```

After 실행 로그 핵심 구간:

```
2026-06-04 06:19:53,040 [INFO] [MemoryWorker] Current Heap: 25MB
2026-06-04 06:19:56,064 [INFO] [MemoryWorker] Current Heap: 50MB
2026-06-04 06:19:59,085 [INFO] [MemoryWorker] Current Heap: 75MB
2026-06-04 06:20:02,095 [INFO] [MemoryWorker] Current Heap: 100MB
2026-06-04 06:20:05,110 [INFO] [MemoryWorker] Current Heap: 125MB
2026-06-04 06:20:08,131 [INFO] [MemoryWorker] Current Heap: 150MB
2026-06-04 06:20:08,131 [CRITICAL] [MemoryGuard] Memory limit exceeded (150MB >= 128MB) /
(Recommend Over 256MB)
2026-06-04 06:20:08,132 [CRITICAL] [MemoryGuard] Self-terminating process 39 to prevent
system instability.
Killed
```

After monitor.csv 요약:

```
first=17096KB at 2026-06-04 06:19:51
last=145116KB at 2026-06-04 06:20:07
max=145116KB at 2026-06-04 06:20:06
```

3. Root Cause Analysis (원인 분석)

MemoryWorker 로그에서 Heap이 25MB 단위로 지속 증가한다. monitor.csv의 RSS도 50MB 제한 실행에서는 약 17MB에서 42MB까지, 128MB 제한 실행에서는 약 17MB에서 145MB까지 증가했다. 메모리 사용량이 제한값 근처에 도달하자 MemoryGuard가 자체 보호 정책으로 프로세스를 종료했다.

OS 관점에서는 프로세스의 힙/가상 메모리 영역이 계속 커지면 물리 메모리 압박이 발생하고, 시스템 전체 안정성이 나빠질 수 있다. 이 애플리케이션은 그 전에 MEMORY_LIMIT 값을 기준으로 자체 종료하여 시스템 전체 OOM으로 확산되는 것을 막도록 설계된 것으로 보인다.

4. Workaround & Verification (조치 및 검증)

임시 조치로 MEMORY_LIMIT을 50MB에서 128MB로 상향했다.

Before/After 비교:

항목	Before	After
MEMORY_LIMIT	50MB	128MB
마지막 Heap 로그	50MB	150MB
최대 RSS 관측값	42,708KB	145,116KB
종료 원인	MemoryGuard	MemoryGuard
생존 시간	약 5초	약 17초

상향 조정 후 프로세스는 더 오래 생존했지만, Heap 증가 패턴 자체는 사라지지 않았다. 근본 해결은 애플리케이션 코드에서 장기 보관 컬렉션, 캐시, 버퍼 등을 해제하거나 상한 정책을 두는 것이다. 운영 임시 조치로는 MEMORY_LIMIT을 서비스 특성에 맞게 256MB 이상으로 설정하고, RSS 증가율을 모니터링해 재시작/알림 정책을 함께 두는 것이 필요하다.

[Bug] CPU Latency - 과도한 CPU 목표 설정으로 CpuWorker 임계치 위반 후 Watchdog성 종료

1. Description (현상 설명)

agent-leak-app을 CPU_MAX_OCCUPY=100으로 실행하면 부트 시퀀스에서 CPU 설정 경고가 표시되고, CpuWorker의 내부 부하 값이 5.00%에서 53.77%까지 상승한 뒤 CPU Threshold Violated 로그와 함께 프로세스가 종료된다.

CPU_MAX_OCCUPY=10으로 낮춘 after 실행에서는 부하가 10%에 도달할 때마다 cooldown으로 전환되며 90초 관찰 시간 동안 임계치 위반 없이 살아 있었다.

2. Evidence & Logs (증거 자료)

Evidence:

- Before: B1-2/evidence/cpu-before/20260604-062432/
- After: B1-2/evidence/cpu-after/20260604-062541/

Before 실행 조건:

```
MEMORY_LIMIT=512
CPU_MAX_OCCUPY=100
MULTI_THREAD_ENABLE=false
```

Before 부트 경고:

```
[ CPU      ] Limit: 100%           [ WARNING: Recommend Under 50% ]
```

Before 실행 로그 핵심 구간:

```
2026-06-04 06:24:34,463 [INFO] [CpuWorker] Started. Maximum CPU Limit: 100%
2026-06-04 06:24:34,464 [INFO] [CpuWorker] Current Load: 5.00%
2026-06-04 06:24:37,584 [INFO] [CpuWorker] Current Load: 9.32%
2026-06-04 06:24:40,706 [INFO] [CpuWorker] Current Load: 18.46%
2026-06-04 06:24:43,833 [INFO] [CpuWorker] Current Load: 26.14%
2026-06-04 06:24:46,957 [INFO] [CpuWorker] Current Load: 29.99%
2026-06-04 06:24:50,084 [INFO] [CpuWorker] Current Load: 31.81%
2026-06-04 06:24:53,214 [INFO] [CpuWorker] Current Load: 39.57%
2026-06-04 06:24:56,335 [INFO] [CpuWorker] Current Load: 40.53%
2026-06-04 06:24:59,453 [INFO] [CpuWorker] Current Load: 46.42%
2026-06-04 06:25:02,574 [INFO] [CpuWorker] Current Load: 53.77%
2026-06-04 06:25:02,680 [CRITICAL] [CpuWorker] CPU Threshold Violated! (53.77%).
Terminated
```

Before monitor.csv / top.log 관찰:

```
ps_cpu_first=8.8% at 2026-06-04 06:24:32
ps_cpu_last=0.9% at 2026-06-04 06:25:02
ps_cpu_max=8.8% at 2026-06-04 06:24:32
```

컨테이너의 ps/top 샘플은 nice=10으로 낮춰진 프로세스의 host-visible CPU 사용률을 낮게 포착했지만, 애플리케이션 Watchdog 판단 기준인 CpuWorker Current Load는 53.77%까지 증가했다.

After 실행 조건:

```
MEMORY_LIMIT=512
CPU_MAX_OCCUPY=10
MULTI_THREAD_ENABLE=false
```

After 실행 로그 핵심 구간:

```
2026-06-04 06:25:44,866 [INFO] [CpuWorker] Started. Maximum CPU Limit: 10%
2026-06-04 06:25:44,866 [INFO] [CpuWorker] Current Load: 5.00%
2026-06-04 06:25:46,983 [INFO] [CpuWorker] Peak reached (10.00%). Starting cooldown...
2026-06-04 06:25:47,991 [INFO] [CpuWorker] Current Load: 10.00%
2026-06-04 06:25:53,232 [INFO] [CpuWorker] Cooldown complete (5.00%). Resuming load
increase...
...
2026-06-04 06:27:12,225 [INFO] [CpuWorker] Current Load: 5.00%
Terminated
```

After result.env:

```
STATE=running_after_timeout
ENDED_AT=20260604-062714
```

Terminated는 Watchdog 종료가 아니라 evidence runner가 90초 스냅샷을 확보한 뒤 종료한 결과다.

3. Root Cause Analysis (원인 분석)

CPU_MAX_OCCUPY=100은 부트 단계에서 권장 범위를 벗어난 값으로 경고된다. 이 설정에서는 CpuWorker가 부하를 계속 높이다가 내부 보호 임계값으로 보이는 50%대를 넘고, CPU Threshold Violated를 기록한 뒤 종료된다.

CPU 과점유는 OS 스케줄러 관점에서 다른 프로세스의 run queue 대기 시간을 늘리고, 같은 호스트의 응답 지연을 유발한다. 이 앱은 CpuWorker 부하가 보호 임계치를 넘으면 프로세스를 종료해 시스템 전체 지연을 줄이는 Watchdog성 보호 조치를 수행한다.

4. Workaround & Verification (조치 및 검증)

임시 조치로 CPU_MAX_OCCUPY를 100%에서 10%로 낮췄다.

Before/After 비교:

항목	Before	After
CPU_MAX_OCCUPY	100%	10%
최대 앱 부하 로그	53.77%	10.00%
보호 동작	CPU Threshold Violated 후 종료	10% 도달 시 cooldown
실행 결과	약 30초 후 종료	90초 관찰 동안 생존

운영 임시 조치로는 CPU_MAX_OCCUPY를 50% 미만으로 제한하고, CPU 부하가 특정 임계치에 접근하면 cooldown 또는 rate-limit이 먼저 동작하도록 설정한다. 근본 해결은 CPU 집약 작업을 작은 단위로 쪼개고, 워커 동시성/큐 처리량에 상한을 두는 것이다.

[Bug] Deadlock - 멀티스레드 모드에서 두 워커가 상대 자원을 기다리며 무응답 상태 지속

1. Description (현상 설명)

MULTI_THREAD_ENABLE=true로 실행하면 부트 단계에서 concurrent mode 경고가 표시된다. 이후 Worker-Thread-1은 Shared_Memory_A를 점유한 상태에서 Socket_Pool_B를 기다리고, Worker-Thread-2는 Socket_Pool_B를 점유한 상태에서 Shared_Memory_A를 기다린다.

마지막 로그가 두 스레드의 WAITING ... BLOCKED 상태에서 멈춘 뒤, 90초 관찰 종료 시점까지 PID는 존재하지만 CPU/MEM 변화와 추가 로그가 없다. 이는 전형적인 circular wait 기반 Deadlock이다.

2. Evidence & Logs (증거 자료)

Evidence:

- Before: B1-2/evidence/deadlock-before/20260604-062841/
- After: B1-2/evidence/deadlock-after/20260604-063035/

Before 실행 조건:

```
MEMORY_LIMIT=512
CPU_MAX_OCCUPY=10
MULTI_THREAD_ENABLE=true
```

Before 부트 경고:

```
[ THREAD ] Concurrency: True          [ WARNING ]
>>> SYSTEM WARNING: POTENTIAL DEADLOCK IN CONCURRENT MODE.
```

Before 실행 로그 마지막 구간:

```
2026-06-04 06:28:48,593 [INFO] [AgentWorker][Worker-Thread-1] LOCK ACQUIRED:
[Shared_Memory_A]. (Holding...)
2026-06-04 06:28:48,593 [INFO] [AgentWorker][Worker-Thread-2] LOCK ACQUIRED:
[Socket_Pool_B]. (Holding...)
2026-06-04 06:28:50,595 [INFO] [AgentWorker][Worker-Thread-1] Need resource [Socket_Pool_B]
to finish job.
2026-06-04 06:28:50,596 [INFO] [AgentWorker][Worker-Thread-1] WAITING for
[Socket_Pool_B]... (Status: BLOCKED)
2026-06-04 06:28:50,604 [INFO] [AgentWorker][Worker-Thread-2] Need resource
[Shared_Memory_A] to write logs.
2026-06-04 06:28:50,604 [INFO] [AgentWorker][Worker-Thread-2] WAITING for
[Shared_Memory_A]... (Status: BLOCKED)
```

Before monitor.csv 처음/마지막 행:

```
first=2026-06-04 06:28:41,38,SNl,6.9,0.2,17108,22288,1,agent-leak-app-
last=2026-06-04 06:30:10,38,SNl,0.0,0.1,13772,169872,3,agent-leak-app-
```

Before process_snapshot.txt:

```
agent          38      33  0 06:28 ?          00:00:00 /work/agent-app-leak/agent-leak-app-
arm64

PID    PPID  STAT %CPU %MEM  RSS    VSZ  NLWP    ELAPSED  COMMAND
38      33  SNl   0.0  0.1 13772 169872   3        01:29  agent-leak-app-

PID    TID  STAT %CPU %MEM  COMMAND
38      38  SNl   0.0  0.1  agent-leak-app-
38     120  SNl   0.0  0.1  agent-leak-app-
38     121  SNl   0.0  0.1  agent-leak-app-
```

After 실행 조건:

```
MEMORY_LIMIT=512
CPU_MAX_OCCUPY=10
MULTI_THREAD_ENABLE=false
```

After 정상 실행 로그:

```
[ THREAD ] Concurrency: False      [ OK ]
2026-06-04 06:30:37,738 [INFO] [Scheduler] Task Scheduler Initialized.
2026-06-04 06:30:37,739 [INFO] [Scheduler] Registered Tasks: ['Thread-A', 'Thread-B',
'Thread-C']
2026-06-04 06:30:38,855 [INFO] [Scheduler] All tasks completed.
```

After result.env:

```
STATE=running_after_timeout
ENDED_AT=20260604-063208
```

3. Root Cause Analysis (원인 분석)

로그상 자원 대기 순서는 다음과 같다.

1. Worker-Thread-1이 Shared_Memory_A를 점유한다.
2. Worker-Thread-2가 Socket_Pool_B를 점유한다.
3. Thread-1은 작업 완료를 위해 Thread-2가 점유한 Socket_Pool_B를 기다린다.
4. Thread-2는 로그 기록을 위해 Thread-1이 점유한 Shared_Memory_A를 기다린다.

이 상태는 Deadlock 4대 조건을 만족한다.

- 상호 배제: 두 자원은 동시에 하나의 스레드만 점유한다.
- 점유 대기: 각 스레드는 이미 하나의 자원을 들고 다른 자원을 기다린다.

- 비선점: 상대 스레드가 잡은 자원을 강제로 빼앗지 못한다.
- 순환 대기: Thread-1 -> Socket_Pool_B -> Thread-2 -> Shared_Memory_A -> Thread-1 순환이 생긴다.

따라서 프로세스는 죽지 않았지만 진행도 하지 못하는 무응답 상태가 된다.

4. Workaround & Verification (조치 및 검증)

임시 조치로 MULTI_THREAD_ENABLE을 true에서 false로 변경했다. 이 설정에서는 concurrent lock 경로가 실행되지 않고, 스케줄러 작업이 정상 완료된다.

Before/After 비교:

항목	Before	After
MULTI_THREAD_ENABLE	true	false
마지막 의미 있는 로그	두 워커 모두 WAITING ... BLOCKED	Scheduler All tasks completed
PID 상태	90초 뒤에도 살아 있음	90초 동안 정상 워크로드 진행
스레드 관측	3개 스레드, CPU 0.0% 정체	3개 스레드, Memory/CpuWorker 계속 진행

근본 해결은 모든 스레드가 동일한 순서로 락을 획득하도록 강제하거나, 락 획득에 timeout/backoff를 적용하고 실패 시 이미 획득한 락을 해제하도록 만드는 것이다.

[Analysis] 로그 패턴 분석을 통한 스케줄링 알고리즘 추론

1. 로그 관찰 개요

정상 실행 케이스(B1-2/evidence/cpu-after/20260604-062541/app.log)에서 Scheduler가 Thread-A, Thread-B, Thread-C를 등록하고 실행한다. 각 스레드는 20%, 40%까지 진행한 뒤 Preempted되고, 이후 다른 스레드가 실행된다.

2. 증거 자료

핵심 로그:

```
2026-06-04 06:25:43,725 [INFO] [Thread-A] Task Started. Calculating... (20%)
2026-06-04 06:25:43,776 [INFO] [Thread-A] Calculating... (40%)
2026-06-04 06:25:43,829 [INFO] [Thread-A] Preempted. Progress saved at (40%)
2026-06-04 06:25:43,880 [INFO] [Thread-B] Task Started. Calculating... (20%)
2026-06-04 06:25:43,935 [INFO] [Thread-B] Calculating... (40%)
2026-06-04 06:25:43,990 [INFO] [Thread-B] Preempted. Progress saved at (40%)
2026-06-04 06:25:44,046 [INFO] [Thread-C] Task Started. Calculating... (20%)
2026-06-04 06:25:44,102 [INFO] [Thread-C] Calculating... (40%)
2026-06-04 06:25:44,156 [INFO] [Thread-C] Preempted. Progress saved at (40%)
2026-06-04 06:25:44,213 [INFO] [Thread-A] Resumed. Calculating... (60%)
2026-06-04 06:25:44,265 [INFO] [Thread-A] Calculating... (80%)
2026-06-04 06:25:44,316 [INFO] [Thread-A] Preempted. Progress saved at (80%)
2026-06-04 06:25:44,370 [INFO] [Thread-B] Resumed. Calculating... (60%)
2026-06-04 06:25:44,421 [INFO] [Thread-B] Calculating... (80%)
2026-06-04 06:25:44,476 [INFO] [Thread-B] Preempted. Progress saved at (80%)
2026-06-04 06:25:44,531 [INFO] [Thread-C] Resumed. Calculating... (60%)
2026-06-04 06:25:44,582 [INFO] [Thread-C] Calculating... (80%)
2026-06-04 06:25:44,636 [INFO] [Thread-C] Preempted. Progress saved at (80%)
2026-06-04 06:25:44,688 [INFO] [Thread-A] Resumed. Calculating... (100%)
2026-06-04 06:25:44,744 [INFO] [Thread-B] Resumed. Calculating... (100%)
2026-06-04 06:25:44,798 [INFO] [Thread-C] Resumed. Calculating... (100%)
2026-06-04 06:25:44,857 [INFO] [Scheduler] All tasks completed.
```

3. 패턴 분석 및 결론

관찰된 실행 순서는 A -> B -> C -> A -> B -> C -> A -> B -> C이다. 하나의 스레드가 100%까지 끝난 뒤 다음 스레드를 시작하지 않고, 40%와 80% 지점에서 선점되어 다른 스레드로 넘어간다.

따라서 FCFS는 아니다. FCFS라면 Thread-A가 완료된 뒤 Thread-B, Thread-C가 순차 실행되어야 한다.

Priority 방식으로 보기도 어렵다. 특정 스레드가 계속 우선 실행되거나 더 많은 CPU 시간을 배정받는 패턴이 없고, A/B/C가 같은 진행률 단위로 반복 실행된다.

최종 결론은 Round-Robin에 가장 가깝다. 각 작업에 작은 time slice가 주어지고, 진행 상태를 저장한 뒤 다음 작업으로 제어권을 넘기는 방식이다.

4. 장단점 및 적합한 아키텍처

Round-Robin의 장점은 작업 간 공정성이 높고, 한 작업이 전체 실행 흐름을 독점하지 않는다는 점이다. 사용자 요청을 번갈아 처리해야 하는 웹 서버, 채팅 서버, interactive agent처럼 응답성이 중요한 서비스에 적합하다.

단점은 context switch 비용이 있고, 처리량만 중요한 배치 작업에서는 FCFS나 우선순위 기반 큐보다 비효율적일 수 있다는 점이다. 또한 time slice가 너무 짧으면 전환 오버헤드가 커지고, 너무 길면 응답성이 떨어진다.

이 앱의 로그 패턴은 장애 분석 미션에서 스레드 선점과 공정성 개념을 보여주기 위한 Round-Robin 시뮬레이션으로 판단된다.